Multi-Agent Debate for Discovering Quantitative Trading Signals

CS153 Final Project · Stanford · June 2026

Patrick Flanagan

Contents

Multi-Agent Debate for Discovering Quantitative Trading Signals

Patrick Flanagan · CS153 Final Project · Stanford · June 2026

What this project is, in one sentence

For my CS153 final project I built an LLM agent system that helps me come up with new ideas for the computer-driven stock-trading strategy I run.

If "computer-driven stock-trading strategy" doesn't mean anything specific to you, the next three sections explain — from scratch — what kind of trading I do, why it's hard, and why an LLM might help. If you already know, skip ahead to **"What I built and why."**

---

1. What kind of trading is this

I don't pick stocks by hand. I run a **systematic quantitative-equity strategy**: every trading day, a computer system I built looks at the universe of about 3,000 US large-cap stocks, processes a lot of data about each one, and produces a ranked list. The system then *buys* the top-ranked stocks (a "long" position) and simultaneously *sells short* the bottom-ranked stocks (a "short" position is a bet that the stock will go down). It holds those positions for a few days, then re-ranks the universe and adjusts the holdings. Rinse and repeat.

Because the strategy holds an equal dollar amount long and short at any given time, it's roughly **market-neutral** — the goal is to make money from the *gap* between the top-ranked and bottom-ranked stocks, regardless of whether the overall stock market goes up or down on any given day. In practice my portfolio's beta to the S&P 500 is **-0.058**, effectively zero.

The whole question, then, is: how does the system decide the daily ranking?

---

2. What "factors" are and how the ranking gets built

A **factor** is just a formula that produces a number for each stock each day. You compute the formula for every stock in the universe, sort the universe by that number, and the *hypothesis* is that the ranking predicts which stocks will outperform.

Simple example: **12-month price momentum**. For each stock, compute its total return over the trailing 12 months. Rank the universe by that number. Historically, the stocks ranked highest by this formula have outperformed the stocks ranked lowest over the next month. That's one factor. My strategy uses several thousand — they look at things like recent returns, financial-statement ratios, trading volume, macroeconomic signals, and combinations of all of the above.

Every trading day, a machine-learning model takes all the factor values for every stock and combines them into one final composite score per stock. That composite score is what determines the long/short ranking.

So the strategy's quality is bottlenecked on **the quality of the factor library**. Better factors → better ranking → better returns.

---

3. Why this is hard, and the metric you'll see

The metric I care about is **Sharpe ratio**: annualised return divided by annualised return volatility. It's a unit-free way of saying "how much edge do you have per unit of risk." A Sharpe of 1 is fine. 2 is rare. 3 is the kind of thing big institutional investors pay attention to. Mine is **3.33** over 4.8 years out of sample at the deployed configuration — that's the headline number, and a lot of this report is about how I got there.

The reason getting there is hard: **most apparent factors are noise.** Tens of thousands of researchers work on this. The vast majority of factor ideas that look great on the historical data they were fit on fail when you test them on data they weren't (the "out-of-sample" test). The entire game is finding the rare ideas that survive honest out-of-sample

testing — and then layering enough of them together that the composite ranking is more robust than any single factor.

To keep finding edges before the existing ones decay, the strategy needs a *constant stream of new factor ideas* to test. That generation of new ideas is the creative bottleneck of the whole pipeline. It's what I spend most of my research time on. And it's exactly the kind of open-ended, "be creative" task that LLMs are supposed to be good at.

What I built and why

So the obvious first move is to just ask an LLM to propose new factor ideas. When I tried that with a single LLM, two things went wrong every time:

- **Every proposal sounded like momentum.** Even with high temperature and "be creative" prompting, a single LLM collapses onto the same handful of textbook ideas.
- **The LLM cheerfully approved its own work.** When the same agent proposed a factor and then reviewed it, the approval rate was close to 100%. That meant my "filter" wasn't filtering anything — proposals with subtle lookahead bias, ignored transaction costs, or in-sample overfitting were sailing through.

So I built a four-stage debate system to test the claim that **mandate-diversified Proposers plus a structurally separated Critic produce better candidate factors than a single LLM asking itself.**

The system, plain English

Four LLM agents, each with their own job:

1. **Proposer.** Five copies of it run in parallel. Each copy gets a different *mandate*: textbook, cross-domain, contrarian, interaction, orthogonal. The system prompt tells each one *"you are one of 5 — do not hedge, commit to your mandate."* That single instruction is what stops everyone collapsing onto momentum.
2. **Critic.** Reviews the proposals in batches. Crucially, the Critic only sees the proposal's name, rationale, and computation description — *never* the Proposer's chain of thought or which slot it came from. That structural separation is what stops the anchoring problem that makes single-agent self-review useless.
3. **Revisor.** Every "revise" verdict from the Critic gets fixed in a single batched LLM call. One call, ~20% of the slate recovered. Cheap.
4. **Ranker.** Only fires when the Critic approves more than the slate cap. Picks for *diversity first*, *relevance second*, *simplicity as tie-breaker*.

The full flow:

```
research challenge
       │
       ▼
Proposer × 5 parallel slots ———> ~25 raw candidates
```

```
(textbook / cross_domain / contrarian / interaction / orthogonal)
       │
       ▼
Critic in batches → {approve, revise, reject}
       │
       ▼
Revisor (one batched call across every "revise")
       │
       ▼
Backfill if approved < target
       │
       ▼
Ranker picks top-k if approved > max
       │
       ▼
final slate of candidate factors → fed into the live strategy's
factor library, which composes them with operators, gates the
results through an FDR significance test, and weights them in
a walk-forward ridge regression that ranks stocks.
```

Each of the four agents has its own system prompt; they're all in `prompts/` as standalone text files. Edit any of them without touching the Python.

A real debate, walked through

The easiest way to see what the system actually does is to look at one debate end-to-end. The repo has three real (redacted) production transcripts in `transcripts/`; this is the most representative one.

The challenge given to the agents: a research scenario where the live ranker had over-rotated into a crowded earnings-growth × momentum trade right before a volatility-regime shift. Propose new candidate factors that would have given the ranker more diverse information on a day like that.

What happened: - The five Proposer slots produced 29 raw candidates between them. - The Critic approved 10, sent 10 back for revision, rejected 0. - The Revisor fixed 4 of the 10 revise verdicts. Final slate: 14. - 13 LLM calls total. Wall clock: 8 minutes.

The interesting part — what the Critic actually caught. One of the textbook-slot proposals was a 60-day rolling-alpha factor. The Critic flagged it:

> *"The proposed derivation uses a 60-day rolling regression on the full window including the target date. This embeds lookahead — the regression coefficient for day t would be fitted on data including day t's return."*

And gave specific revision guidance:

> *"Shift the rolling window to end at t-1, not t. Use the coefficient from the regression on days [t-60, t-1] to compute the alpha for day t."*

That's the kind of catch a careful human reviewer would make. A single LLM proposing and reviewing its own work would not. The Revisor took the guidance, fixed the factor, and the revised version was approved.

You can replay this exact debate locally with no API key: `python run_demo.py --dry-run`.

Did the system work?

After two months in production:

Stage	Count	Rate
Raw LLM proposals	**1,214**	(baseline)
Survived Critic + Revisor + validation	**448**	**36.9%**
Went on to seed at least one downstream factor	360	80% of survivors
Went on to seed at least one *promoted* downstream factor	357	80% of survivors
Promoted downstream factors total	**800**	

The 36.9% number is the empirical fingerprint. A single LLM agent that reviews its own work approves at near 100%. The fact that this system rejects roughly two thirds of raw LLM proposals — through the combination of adversarial Critic and the validation gate that follows — is the architecture doing real work. It's filtering out the lookahead bugs, the duplicates of things already in the library, and the proposals that double down on whatever direction just failed.

Per-debate runtime is steady:

Metric	Min	Median	Max
Raw proposals per debate	16	27	35
Approved per debate	11	14	15
LLM calls per debate	8	13	16
Wall clock (seconds)	270	535	1,191

Notice that LLM call count stays roughly constant as raw-proposal count grows. That's the batched Critic and single-call Revisor working as designed — they don't scale linearly with the number of proposals.

The strategy's factor library now consists of **783 LLM-proposed factors and 89 hand-curated ones** — about 90% authored by this system.

Where this fits in the live trading strategy

I want to be clear about what the LLM is and isn't doing.

The debate system sits at the top of the strategy's research pipeline. It produces candidate factor ideas. Each idea then runs through three more stages before any of it touches the live portfolio:

1. **Factor builder** — composes each candidate with standard quant operators (ratios, differences, rolling stats) and with other factors, expanding the search space.
2. **Significance gate** — pre-registered false-discovery-rate control rejects combinations that look good by chance. Because the thresholds are set before the data is touched, the search space can't be turned into a p-hacking machine.
3. **Ridge ensemble** — ranks the survivors cross-sectionally each day and weights them by their recent out-of-sample information coefficient.

The long top-K / short bottom-K portfolio is then built from the ridge's composite score. The LLM contributes to step 1. The portfolio numbers below reflect all four stages working together — I'm not claiming the LLM is solely responsible for them. What I am claiming is that the LLM debate is the source of about 90% of the factor library that every downstream stage operates on.

The strategy's actual performance

Out-of-sample evaluation window: **2021-07-16 → 2026-05-05** (1,206 trading days, **4.8 years**). Walk-forward training, per-symbol transaction cost gating, sanity-validated against a canonical snapshot. The deployed configuration is **K = 25 at 1.5× gross leverage**.

K	Unlevered CAGR	Unlevered Sharpe	Unlevered Max DD	1.5× CAGR	1.5× Sharpe	1.5× Max DD	1.5× total return
5	84.1%	2.77	-15.1%	142.7%	2.74	-25.2%	69.6×
10	87.6%	3.16	-12.5%	141.1%	3.02	-20.8%	67.5×
25	**81.1%**	**3.43**	**-10.0%**	**134.0%**	**3.33**	**-15.7%**	**58.4×**
50	70.9%	3.45	-8.6%	119.9%	3.44	-12.9%	43.4×
100	58.8%	3.44	-7.3%	98.6%	3.44	-10.8%	26.6×
250	42.2%	3.32	-6.3%	70.4%	3.40	-9.3%	12.8×

The deployed row — K=25 at 1.5× leverage — is the headline. On that configuration: Newey-West t-statistic **8.17**, SPY beta **-0.058** (effectively market-neutral), 5-day non-overlapping hit rate **71.8%**, **48 of 59 OOS months positive**, **$1 → $58.40** over the 4.8 years.

Sharpe is essentially flat across K = 10 to 250 at ~3.3 to 3.45, which means the signal isn't living in one weird corner of the rank distribution. CAGR scales sub-linearly with leverage (134% at 1.5× versus 81% unlevered = 1.65× CAGR for 1.5× leverage), which is what you'd expect once larger drawdowns start hurting compounding.

What I learned about LLM agents

1. **Mandate diversification matters more than temperature.** I tried high-temperature single-Proposer setups first. Everything still collapsed to momentum. Five parallel Proposers, each told *"you are one of five — do not hedge"*, produce candidate sets that span the signal space. It's not a sampling-temperature problem; it's a prompt-structure problem.
2. **Structurally separating the Critic from the Proposer is the load-bearing decision.** Giving the Critic only `(name, rationale, derivation)` — never the Proposer's chain of thought, never which slot the proposal came from — is what changes the approval rate from ~100% to 37%. Without that separation, the Critic anchors on the Proposer's framing.
3. **A batched Revisor is the cheapest filter you can add.** One LLM call, ~20% of the slate recovered. Re-proposing rejected candidates from scratch would cost five times as much.
4. **The Ranker fires occasionally but is load-bearing when it does.** On clean-pass days where the Critic approves more than the slate cap, the Ranker is the only thing preventing the slate from collapsing onto whichever Proposer slot got lucky that round.
5. **The reliability engineering around the agent logic is harder than the agent logic itself.** The continuous-debate cron has logged 9,592 round-starts; only 5 completed cleanly. The agent code works — the wrapper around it (rate-limit handling, dependency pinning, paid-tier fallback, alerting when the cron drifts) is where time disappears in production. The 1,214 proposals above were produced in a ~3-day window in early April before the cron drifted; reliability is what's keeping the system from producing another batch right now.

What's broken and what I'd do next

- **The continuous-debate cron has been broken since April** due to a module-path drift in the wrapper. Ten-minute fix that I haven't prioritised — the existing library is enough to keep the strategy fed for now.
- **I don't have a formal single-agent ablation yet.** The 37% materialisation rate strongly suggests the architecture is earning its complexity, but a clean three-way head-to-head — single-agent vs. no-Critic vs. full debate on identical challenges — would actually pin down how much each piece is contributing.
- **No per-factor attribution to portfolio P&L yet.** I quote the strategy's Sharpe of 3.33 as integrated-system evidence, not as marginal LLM contribution. The infrastructure for a leave-LLM-factors-out backtest exists; wiring it up is a separate project.
- **The Critic shares a model family with the Proposer.** A cross-vendor Critic — different model, different training data — would be a meaningfully stronger structural separation. Worth testing next.

How to run this

```
pip install -r requirements.txt

# Replay one of the recorded production debates — no API key needed
python run_demo.py --dry-run

# Run one fresh debate end-to-end (~5 minutes against free-tier OpenRouter)
export OPENROUTER_API_KEY=...
python run_demo.py

# Walkthrough notebook with executed cells
jupyter notebook demo.ipynb
```

What's in the repo: the four agent classes (`agents.py`), the orchestrator (`debate.py`), all the system prompts (`prompts/`), three real redacted production debates (`transcripts/`), the operational metrics behind the funnel (`operational_metrics.json`), and the walk-through notebook (`demo.ipynb`).

What's *not* in the repo: the downstream factor builder, the FDR significance gate, the ridge ensemble, the backtest engine, or the actual factor formulas that the strategy trades. Those stay in the production codebase. The portfolio numbers above are evidence that the LLM debate system feeds something that works; they aren't reproducible from this scaffold alone, and I tried to make sure the README and this report don't suggest otherwise.